Namespaces Without a Name Server

Freshness by communication, and a fully abstract encoding of the π-calculus in the rho calculus

L. G. Meredith*

August 2026

Abstract

The rho calculus has no name-restriction operator and no ambient supply of atoms: names are quoted processes. An encoding of the π-calculus into it is therefore a test of a specific design objective — that ν can be discharged without positing a runtime source of fresh names. The encoding published with the calculus in 2005 was intended to meet that objective and does not: Lybech exhibited two counterexamples and gave a corrected encoding in which freshness is supplied by a central name server reached on a fixed channel. That encoding is correct. It meets a different design objective, and Lybech says as much in his conclusion, where he describes a per-replication local server as closer to the original intention and leaves it open on account of a scaffolding problem: to build a source of fresh names one must already have one.

We settle the scaffolding problem. Three facts about the rho calculus do the work. First, a reduction adds at most one name to a term, so producing n distinct names costs n rendezvous, and no translation can write them. Second, substitution reaches the object of a lift and does not reach beneath a quote, so a name derived from a runtime parameter must be built by communication — one quote level per rendezvous. Third, the rho duplicator reproduces quoted code verbatim, so copies of a replicated body cannot be told apart by their syntax. Together these force address arithmetic out of the translation and into a process, and they determine that process almost uniquely: a two-communication allocator we call Fresh_2.

The architecture Fresh_2 realises is Abramsky's. His proof expressions for classical linear logic carry an invariant — names are a root together with a binary address, and addresses sharing a root are incompatible in the prefix ordering — which is exactly the invariant the 2005 encoding required and never stated, and which its splitting of parameters across parallel composition already maintains. What it omitted is the other half of the Copy rule: the clause that re-contracts a copied box's side ports so that the parent's namespace is never re-rooted. Abramsky's Copy cannot be transcribed, because it renames the contents of a box at the metalevel and rho has no such operation; the realisation here is therefore different in kind, and is the technical content of this note.

We give the encoding, prove that its allocation traffic forms a tree isomorphic to the parse tree of the source — so that no channel is shared by two allocation sites, which is the precise sense in which it is not a distributed name server — verify it against both counterexamples, and prove it fully abstract for a bisimilarity whose observers are the encodings of source contexts. That restriction is forced: an unrestricted rho context separates the encodings of P and $P \mid \mathbf{0}$.

*F1R3FLY.io. lgreg.meredith@f1r3fly.io

1 Introduction

Process calculi customarily posit a countably infinite set of atomic names and an operator $(\nu x)P$ producing one guaranteed fresh. An implementation must discharge both, and in a distributed setting the discharge is not routine.

The rho calculus [11] moves the question inside the theory. Names are quoted processes; the only binder is input; there is no ν. Whether this is a viable position is the question of whether the π-calculus can be encoded into it without reintroducing what was removed, and the original paper offered an encoding to that end. Its design objective can be stated in one line:

no process in the image of the translation is a source of names for any other.

The price paid was that the translation compiles a whole term at a time rather than acting clause by clause on a signature, because the namespace discipline is a property of a term's structure.

Lybech [10] showed that the encoding does not work.[1] He gives two counterexamples, diagnoses their common cause exactly, supplies a new encoding, and proves it valid against criteria adapted from Gorla [6] together with a criterion of *parameter independence* which is the right addition for parametrised translations. None of that is in dispute here, and his diagnosis is adopted below without amendment.

His encoding obtains freshness from a process

$$!N(x, z, v, s)$$

reached on a fixed channel v that every occurrence of ν in the term must contact. That is a different implementation assumption — a global freshness oracle — rather than a different source language, and under that assumption his results stand. It is also an assumption he declines to be comfortable with: his conclusion observes that for a distributed system whose topology is not a star the translation does not represent the source's structure, proposes instead that each replication instantiate its own source of names, calls this closer to the intention of the 2005 encoding, and leaves it open, because such a scheme appears to need a source of fresh names in order to build one.

The idea, in four lines

There is no regress. Write $\mathsf{L}(\sigma) = @(\sigma!(\mathbf{0}))$ and $\mathsf{R}(\sigma) = @(\mathsf{for}(\, y \leftarrow \sigma\,)\mathbf{0})$ for two structurally distinct names built from σ. Then

$$\mathsf{Fresh}_2(\sigma;\, u,\, v).\, P \;\triangleq\; \mathsf{for}(\, u \leftarrow \sigma\,)\mathsf{for}(\, v \leftarrow \sigma\,)P \;\mid\; \sigma!(\sigma!(\mathbf{0})) \;\mid\; \sigma!(\mathsf{for}(\, y \leftarrow \sigma\,)\mathbf{0})$$

binds u and v to $\mathsf{L}(\sigma)$ and $\mathsf{R}(\sigma)$ in two communications, and it does so *when σ is a name received at runtime*, because the two lifts mention σ in their objects, which substitution reaches, rather than under quotes, which it does not. A name allocator is thus not a service to be contacted but a two-message protocol that any node can run on a name it already holds. The whole encoding is this construction applied at each node of the source term to the address that node was given by its parent; the regress terminates at a seed computed by the translation from the free names of the term and consumed once.

Everything else in this note is an account of why Fresh_2 has the shape it has, why nothing simpler will do, and what it buys.

[1] The author discussed the matter with Lybech in early 2023 and suggested that Abramsky's construction [1], which had informed the original design, also indicated the repair; the thread was not pursued. The present note makes the argument in full.

Contributions

(i) **Three facts about the rho calculus** (§3), stated for the calculus rather than for any encoding. A reduction adds at most one name to a term (Theorem 3.3), and raises the maximum quote depth of names in play by at most one (Theorem 3.5); and substitution reaches the object of a lift but not the interior of a quote (Lemma 3.7). Between them these say that names are a *metered* resource: n distinct new names cost n rendezvous, and a name k quote levels above what a process holds costs k.

(ii) **A no-go result** (Corollary 7.3): no translation of the 2005 shape can maintain distinctness across replication, since a translation emits a finite term while replication requires unboundedly many distinct names, and the duplicator reproduces quoted code verbatim so that copies cannot be differentiated by syntax. Address arithmetic must be communication.

(iii) **The discipline and its minimal realisation** (§7). Admissibility is a one-line syntactic side condition on translation clauses which both broken clauses violate by inspection; and any admissible allocator costs at least two communications and two lifts mentioning its parameter, which Fresh_2 attains (Proposition 7.6).

(iv) **The encoding** (§8), in which every replication node carries a local address server derived from its own address. Whole-term compilation survives: the allocator tree is static and isomorphic to the parse tree; only the addresses are runtime values.

(v) **A locality theorem** (§9) distinguishing this from a distributed name server. The allocation traffic of $[\![P]\!]$ forms a tree isomorphic to the parse tree of P; no name is the subject of allocations belonging to two distinct nodes; and there is in particular no channel on which every allocation occurs. A server-based encoding has, by contrast, a star.

(vi) **Full abstraction** (§11) for a context-labelled bisimilarity whose observers are the encodings of source contexts, together with a proof (§11.5) that the restriction is forced rather than convenient.

Relation to Abramsky's construction. The architecture is his. The 2005 encoding already implemented one half of it — the splitting of a namespace across a multiplicative branch — and the invariant that half maintains is his, not a weaker one invented for the occasion (§6, Proposition 7.2). The other half, Copy's re-contraction of a copied box's side ports, is what was missing. But it cannot be transcribed: Copy renames the contents of a box at the metalevel, and rho has no metalevel operation on quoted code — this is not an oversight in the calculus but the content of reflection, and the same fact on which Lybech's separation theorem turns. Abramsky supplies the invariant and the architecture; the realisation is necessarily different, and it is the realisation that is at issue here.

Scope. The source language is the asynchronous, choice-free π-calculus with input-guarded replication. Unguarded replication is not implementable — it runs away, and the 2005 encoding of it diverges unconditionally. Lybech restricts to the same fragment for the same reason, and notes that the restriction would not by itself have prevented either counterexample. Cost accounting is out of scope.

2 The rho calculus

Definition 2.1 (Syntax)**.** Processes P and names x are given by mutual recursion:

$$P \;::=\; \mathbf{0} \;\mid\; P_1 \mid P_2 \;\mid\; x!(P) \;\mid\; \mathsf{for}(\, y \leftarrow x \,)P \;\mid\; *x \qquad\qquad x \;::=\; @P$$

$x!(P)$ is the *lift*: it quotes P, creating the name $@P$, and outputs it on x. $\mathsf{for}(\, y \leftarrow x \,)P$ is input, binding y in P; it is the only binder. $*x$ is the *drop*: it strips the quotes from x and runs the process inside.

Definition 2.2 (Name equivalence, structural congruence, reduction)**.** $\equiv_{\mathcal{N}}$ is the least equivalence on names with $@P_1 \equiv_{\mathcal{N}} @P_2$ whenever $P_1 \equiv P_2$, and $@(*x_1) \equiv_{\mathcal{N}} x_2$ whenever $x_1 \equiv_{\mathcal{N}} x_2$; $\equiv$ is the least congruence containing α-equivalence and the abelian monoid laws for $\mid$ with unit $\mathbf{0}$, names being compared up to $\equiv_{\mathcal{N}}$. Reduction is

$$\frac{x_1 \equiv_{\mathcal{N}} x_2}{\mathsf{for}(\, y \leftarrow x_1 \,)P_1 \;\mid\; x_2!(P_2) \;\rightarrow\; P_1\{@P_2/y\}}$$

closed under $\mid$ and $\equiv$.

Substitution is capture-avoiding and replaces all names $\equiv_{\mathcal{N}}$-equal to the substituted variable. Two clauses are used throughout:

$$(@P)\{x/y\} = \begin{cases} x & \text{if } @P \equiv_{\mathcal{N}} y \\ @P & \text{otherwise} \end{cases} \qquad (*x)\{@P/y\} = \begin{cases} P & \text{if } x \equiv_{\mathcal{N}} y \\ *x & \text{otherwise} \end{cases}$$

Notation 2.3. $x!(*z)$ transmits the *name* z along x, since $@(*z) \equiv_{\mathcal{N}} z$. $\mathsf{D}(x) \triangleq \mathsf{for}(\, y \leftarrow x \,)(*y \mid x!(*y))$ is the duplicator; it satisfies $x!(Q) \mid \mathsf{D}(x) \rightarrow Q \mid x!(Q)$. qd denotes Lybech's quote depth: $\mathrm{qd}(@P) = \mathrm{qd}(x)$ if $P \equiv *x$, and $1 + \mathrm{qd}(P)$ otherwise, with $\mathrm{qd}(P)$ the maximum of qd over the names occurring in P, or 0 if there are none.

2.1 Why this notation

The 2005 papers wrote input as $x(y).P$ and quotation with corner quotes. We use $\mathsf{for}(\, y \leftarrow x \,)P$, $@P$, $*x$, for four reasons, the last of which is the one that earns its place in a paper about a bug.

(1) It is ASCII; every term displayed here is a term of rholang, and Appendix C runs.

(2) It extends to joins without renegotiation. Simultaneous receipt is $\mathsf{for}(\, y_1 \leftarrow x_1 \;\&\; y_2 \leftarrow x_2 \,)P$, with & joining within a group and ; separating alternatives. A prefix notation $x(y).P$ has no comparable extension: the prefix is unary by construction. The encoding below does not need joins; the generalisation to polyadic π (§14) does, and Remark 8.7 shows joins removing a nondeterminism that the monadic version has to reason about.

(3) It is the for-comprehension of Scala, Haskell's `do`, and LINQ. A reader with no background in process calculi reads $\mathsf{for}(\, y \leftarrow x \,)P$ correctly at first sight: draw a y from x, then do P. This is not an analogy; RSpace, the store beneath the rho calculus implementations, is a comprehension over a tuplespace.

(4) $@\cdot$ and $*\cdot$ are visibly *operators occupying positions in a term.* Lemma 3.7 is a statement about which positions substitution can reach — it reaches the object of a lift, and not the interior of a quote — and the whole difference between the broken encoding and the repaired one is the difference between $c!(n!(\mathbf{0}))$ and $@(n!(\mathbf{0}))$. In corner quotes that difference is a change of typographic register.

Remark 2.4. "rho" abbreviates **r**eflective **h**igher **o**rder, and the pun — that it follows π — is the point of the name. We do not write it with the letter ρ.

3 Three facts about name creation

This section is about the rho calculus and mentions no encoding. Its content is that names are a metered resource.

Definition 3.1 (Names in name position)**.** For a closed process P, let $\mathcal{N}ms(P)$ be the set of names, up to $\equiv_{\mathcal{N}}$, occurring in name position, defined by

$$\begin{aligned} \mathcal{N}ms(\mathbf{0}) &= \emptyset & \mathcal{N}ms(P_1 \mid P_2) &= \mathcal{N}ms(P_1) \cup \mathcal{N}ms(P_2) \\ \mathcal{N}ms(x!(Q)) &= \{x\} \cup \mathcal{N}ms(Q) & \mathcal{N}ms(\mathsf{for}(\, y \leftarrow x\,)Q) &= \{x\} \cup (\mathcal{N}ms(Q) \setminus \{y\}) \\ \mathcal{N}ms(*x) &= \{x\}. \end{aligned}$$

Note that $@Q$ is not in $\mathcal{N}ms(x!(Q))$: the lift's object is a process, and the name it will become does not exist until the lift fires.

Lemma 3.2. *$P \equiv Q$ implies $\mathcal{N}ms(P) = \mathcal{N}ms(Q)$, and $\mathcal{N}ms(P\{@Q/y\}) \subseteq (\mathcal{N}ms(P) \setminus \{y\}) \cup \mathcal{N}ms(Q) \cup \{@Q\}$.*

Proof. The first by inspection of the axioms of $\equiv$. The second by structural induction on P. The only cases that are not immediate are $*y$, where $(*y)\{@Q/y\} = Q$ and $\mathcal{N}ms(Q)$ appears on the right; and a name position occupied by y, which becomes $@Q$. □

Theorem 3.3 (A reduction creates at most one name)**.** *If $P \to P'$ then $\mathcal{N}ms(P') \subseteq \mathcal{N}ms(P) \cup \{@P_2\}$, where P_2 is the object of the lift consumed by the reduction. In particular $|\mathcal{N}ms(P')| \leq |\mathcal{N}ms(P)| + 1$.*

Proof. By Lemma 3.2 we may work up to $\equiv$ and assume $P = C[\,\mathsf{for}(\, y \leftarrow x_1\,)P_1 \mid x_2!(P_2)\,]$ with $P' = C[\,P_1\{@P_2/y\}\,]$. The context contributes the same names to both sides. For the redex, $\mathcal{N}ms(\mathsf{for}(\, y \leftarrow x_1\,)P_1 \mid x_2!(P_2)) = \{x_1, x_2\} \cup (\mathcal{N}ms(P_1) \setminus \{y\}) \cup \mathcal{N}ms(P_2)$, and by Lemma 3.2 $\mathcal{N}ms(P_1\{@P_2/y\}) \subseteq (\mathcal{N}ms(P_1) \setminus \{y\}) \cup \mathcal{N}ms(P_2) \cup \{@P_2\}$. □

Corollary 3.4 (Freshness costs communication)**.** *If $P \to^k P'$ then $|\mathcal{N}ms(P') \setminus \mathcal{N}ms(P)| \leq k$. Hence a process that exhibits n pairwise non-$\equiv_{\mathcal{N}}$ names not among those it starts with performs at least n reductions.*

Theorem 3.5 (A reduction adds at most one quote level)**.** *If $P \to P'$ then* $\max\{\mathrm{qd}(n) : n \in \mathcal{N}ms(P')\} \leq \max\{\mathrm{qd}(n) : n \in \mathcal{N}ms(P)\} + 1$.

Proof. By Theorem 3.3 the only name in $\mathcal{N}ms(P')$ possibly absent from $\mathcal{N}ms(P)$ is $@P_2$. If $P_2 \equiv *x$ then $@P_2 \equiv_{\mathcal{N}} x \in \mathcal{N}ms(P)$ and there is nothing to prove. Otherwise $\mathrm{qd}(@P_2) = 1 + \mathrm{qd}(P_2) = 1 + \max\{\mathrm{qd}(n) : n \in \mathcal{N}ms(P_2)\}$, and $\mathcal{N}ms(P_2) \subseteq \mathcal{N}ms(P)$. □

Corollary 3.6 (One quote level per rendezvous)**.** *Producing a name whose quote depth exceeds by d the maximum quote depth of the names a process holds requires at least d reductions.*

Lemma 3.7 (Substitution transparency)**.** *The object of a lift is a substitution-transparent position; a quote is not:*

$$(c!(P))\{x/y\} \;=\; c\{x/y\}!((P\{x/y\})) \qquad \textit{whereas} \qquad (@P)\{x/y\} \in \{x,\ @P\}.$$

Consequently a name may be derived from a runtime-bound σ by the process $c!(\sigma!(\mathbf{0}))$, which transmits $\mathsf{L}(\sigma)$ on c; and it may not be derived by writing $@(\sigma!(\mathbf{0}))$ in a term, since that expression is closed to substitution for σ.

Proof. Substitution is defined by structural recursion through process constructors, and the lift's object is a process. Quotation is not a process constructor but the name constructor, and its clause is the two-case clause displayed above. □

Remark 3.8. Theorem 3.3 and Lemma 3.7 pull in opposite directions and together determine the shape of any name discipline in the calculus. The first says that a name cannot be had for free: it is paid for by a rendezvous. The second says where the payment must be made: in the object of a lift, since that is the only position in which a term mentioning a runtime parameter can still be reached by the substitution that instantiates it.

4 The 2005 encoding and its objective

Fix an injection φ from π names to rho names; we suppress it. The static quoting techniques generate from a name σ the *increments*

$$\mathsf{L}(\sigma) = @(\sigma!(\mathbf{0})), \qquad \mathsf{R}(\sigma) = @(\mathsf{for}(\, y \leftarrow \sigma\,)\mathbf{0}),$$

written $+\sigma$ and $\sigma+$ in the original. The translation $[\![-]\!]_{n,p}$ takes two name parameters:

$$\begin{aligned}
[\![\mathbf{0}]\!]_{n,p} &= \mathbf{0} \\
[\![x(y).P]\!]_{n,p} &= \mathsf{for}(\, y \leftarrow x\,)[\![P]\!]_{n,p} \\
[\![\bar{x}\langle z\rangle]\!]_{n,p} &= x!(*z) \\
[\![P_1 \mid P_2]\!]_{n,p} &= [\![P_1]\!]_{\mathsf{L}(n),\mathsf{L}(p)} \mid [\![P_2]\!]_{\mathsf{R}(n),\mathsf{R}(p)} \\
[\![(\nu z)P]\!]_{n,p} &= \mathsf{for}(\, z \leftarrow p\,)[\![P]\!]_{\mathsf{L}(n),\mathsf{L}(p)} \mid p!(*n) \\
[\![!P]\!]_{n,p} &= a!\big(\big(\mathsf{for}(\, n \leftarrow b\,)\mathsf{for}(\, p \leftarrow c\,)([\![P]\!]_{n,p} \mid \mathsf{D}(a) \mid \cdots)\big)\big) \mid \mathsf{D}(a) \mid \cdots
\end{aligned}$$

with $a = n \cdot p$, $b = \mathsf{R}(n)$, $c = \mathsf{R}(p)$, and top-level parameters computed from $\mathrm{fn}(P)$ so that no name of P is derivable from them.

Two features are the design and should be recorded before the encoding is criticised. First, no process in the image is a source of names for any other: the top-level parameters are computed by the translation and never requested. Second, the translation is parametrised by a position in the term rather than by a head constructor, which is the price of the first and the reason it is not a signature homomorphism.

The defect, in the form we shall need, is that the encoding carries an invariant on (n, p) which is nowhere written down.

5 The counterexamples and the name server

Example 5.1 (Splitting). For arbitrary Q, $P_1' = !\big((\nu z)\bar{u}\langle z\rangle \mid Q\big)$ and $P_2' = (\nu z)!\big(\bar{u}\langle z\rangle \mid Q\big)$ are separated in π by a context that receives twice on u and tests the two names for equality; their images are not separated. The replication clause rebinds n and p at each unfolding, but the clause for $\mid$ has already frozen $\mathsf{L}(n)$ and $\mathsf{L}(p)$ inside quotes, where the substitution cannot reach.

Example 5.2 (Nesting). $P_1 = !(\nu z)\bar{u}\langle z\rangle$ and $P_2 = !(\nu q)(\nu z)\bar{u}\langle z\rangle$ are structurally congruent; their images are separated, because the clause for (νq) increments the parameters statically before the clause for (νz) sees them.

Lybech's diagnosis is exact and we adopt it: the property of being "the names most recently replicated" is not preserved by the clause for parallel composition, because the increments that clause writes are static, placing the parameter under a quote, and substitution does not recur under quotes.

Remark 5.3. Both examples use unguarded replication, which our fragment excludes; Lybech records that the restriction would not have prevented either. We restate them in the input-guarded fragment in §10.

The name server. Lybech's encoding is $[\![P]\!]_{n,v} \mid !N(x,z,v,s)$, where

$$!N(x,z,v,s) \triangleq \mathsf{D}(x) \mid x!\big(\big(\mathsf{for}(\,a \leftarrow z\,)\mathsf{for}(\,r \leftarrow v\,)(\mathsf{D}(x) \mid r!(*a) \mid z!(a!(\mathbf{0})))\big)\big) \mid z!(*s)$$

holds a seed on z and answers requests on the fixed channel v with successive left increments. The parameter n is threaded forward and never re-bound, which removes the second error; access to the name source is by a channel that never changes, which removes the first. The encoding is proved valid against compositionality, substitution invariance, operational correspondence, observational correspondence, divergence reflection and parameter independence.

Two observations about it, both of which we use.

(1) The implementation assumption is a global freshness oracle: one process, reached on one fixed channel, is the source of every fresh name in the term. The source language and the correctness results are not at issue; the assumption is what differs from the objective of §1, and §9 makes the difference a property of the image rather than a matter of emphasis.

(2) The server builds its successor name by $z!(a!(\mathbf{0}))$, where a was *received*. This computes $\mathsf{L}(a)$ at runtime and works for the reason given in Lemma 3.7. It is the manoeuvre the broken clauses needed. Deployed once inside a server it looks like an implementation detail; §7 makes it a discipline.

6 Abramsky's construction

Abramsky's computational interpretation of classical linear logic [1] assigns *proof expressions* $\Theta; \bar{t}$ to sequent proofs and gives them a chemical-abstract-machine semantics. Four features matter.

Names carry addresses. A bijection $\mathcal{N} \cong \mathbb{N}_0 \times \{l,r\}^*$ makes a name a pair $\langle x_0, s\rangle$ of a root and a binary address; the variants t^l, t^r extend every address in t by l resp. r. The invariant established by the proof-expression assignment and maintained by every transition is: *for distinct names $\langle x_0, s\rangle, \langle y_0, t\rangle$ in P, $x_0 = y_0$ implies s and t are incompatible in the prefix ordering.*

Contraction is a term former. $\dfrac{\vdash \Theta;\Gamma,\ t{:}?A,\ u{:}?A}{\vdash \Theta;\Gamma,\ t \mathbin{@} u : ?A}$ — the joiner is a term, not a rewrite, and can be embedded and cut against other terms.[2]

Copy.

$$\bar{x}(P) \perp u \mathbin{@} v \ \longrightarrow\ \bar{x} \perp (\bar{x}^{\,l} \mathbin{@} \bar{x}^{\,r}), \quad \bar{x}(P)^{l} \perp u, \quad \bar{x}(P)^{r} \perp v.$$

Copying is demand-driven: the box duplicates only when a contraction asks for two, and Copy is the only rule that increases the size of a proof expression. Distinctness of the copies is by address extension, not allocation: no fresh root is created, and roots are introduced only by the name convention at Axiom, With and Of Course — statically, finitely often. The first clause is the rejoin: the box's side ports are cut against a newly built contraction of the two copies' ports, so the environment continues to see one port and *the parent's namespace is never re-rooted.*

What transfers and what does not. The address structure and the invariant transfer, and the 2005 encoding already used them. The rejoin is what was missing, and it does not transfer as a rule: $(\cdot)^l$ and $(\cdot)^r$ rewrite the interior of a box, and by Lemma 3.7 the rho calculus has no such operation — transmission of code without modification is what reflection is. What can be carried over is the *shape* of the rejoin: a port shared by the copies, with the routing rather than the syntax doing the distinguishing. That is what §8 builds.

PE_2	2005 encoding	
$\mathcal{N} \cong \mathbb{N}_0 \times \{l, r\}^*$	L, R	used
variants t^l, t^r	the clause for $\mid$	used
prefix-incompatibility invariant	—	not stated
Contraction $t \mathbin{@} u$; Copy's first clause	—	absent
demand-driven copying	eager replication (diverges)	absent

7 Diagnosis

Proposition 7.1 (The reconstructed invariant is the wrong one)**.** *Let $I(n,p)$ hold when (n,p) are the names most recently produced by the enclosing replication context. No splitting clause preserves I: a clause delegating to sub-translations at parameters derived from (n,p) gives each sub-translation a pair which is a function of the most recent output and not that output.*

Proposition 7.2 (Prefix-incompatibility is preserved by splitting)**.** *Let* $\mathrm{Inc}(S)$ *hold when the addresses in S are pairwise incompatible in the prefix ordering. If* $\mathrm{Inc}(S)$ *and $\sigma \in S$ then* $\mathrm{Inc}\big((S \setminus \{\sigma\}) \cup \{\sigma l, \sigma r\}\big)$.

Proof. $\sigma l, \sigma r$ are incompatible with each other; and any $\tau \in S \setminus \{\sigma\}$ is incompatible with σ, hence with every extension of σ. □

Taken together these say that the clause for parallel composition is not the defect. Against the invariant the construction's ancestry supplies, splitting is precisely the operation that maintains it. Against the invariant the encoding was actually trying to maintain, nothing does.

[2] Printed in [1] with $u{:}?B$; comparison with the intuitionistic rule and with case (12) of the realizability proof shows this to be a typo for $?A$.

Corollary 7.3 (No static address assignment)**.** *There is no translation with finitely many clauses, each emitting finitely many quoted names, for which the name generated by a ν in the k-th unfolding of a replication is written by the translation, for all k.*

Proof. The image of a fixed term is a finite term, so by Theorem 3.3 the names it exhibits after k reductions number at most $|\mathcal{N}ms([\![P]\!])| + k$, and by Corollary 3.4 n distinct new names require n rendezvous. Hence they cannot all be written; they must be produced. The remaining possibility is that a fixed piece of syntax is differentiated across copies by substitution — but the rho duplicator reproduces the quoted body verbatim, and by Lemma 3.7 substitution does not reach beneath the quote in which that body sits. □

Definition 7.4 (Admissibility)**.** A translation clause is *admissible* if every parameter bound at runtime occurs in it only (a) in subject position of an input or a lift, or (b) within the object of a lift; and never within a quote written by the translation.

Proposition 7.5. *The clauses of §4 for $|$ and for ν are inadmissible; those for $\mathbf{0}$, input and output are admissible.*

Proof. By inspection: $\mathsf{L}(n) = @(n!(\mathbf{0}))$ places the parameter under a written quote, and the two clauses use it. The remaining clauses mention the parameters only by passing them on. □

Proposition 7.6 (Admissible allocation is essentially Fresh_2)**.** *Let A be an admissible process whose only free name is a parameter σ bound at runtime, and suppose A has a reduction sequence after which two names $u \not\equiv_{\mathcal{N}} v$, neither in $\mathcal{N}ms(A)$, are bound in its continuation. Then*

(i) A performs at least two reductions; and

(ii) A contains at least two lifts whose objects mention σ.

Fresh_2 attains both bounds.

Proof. (i) is Corollary 3.4. For (ii), by Theorem 3.3 each of the two names is $@Q$ for the object Q of some lift of A, and the two lifts are distinct since the names differ. Suppose one such Q does not mention σ. Then Q is closed, so $@Q$ is a constant of the translation, the same in every instance of A; two instances of A at distinct parameters would allocate a common name, contradicting the requirement that the allocated names not be among those already held once A is placed in parallel with another instance of itself — which is exactly the situation inside a replicated body. Hence both objects mention σ. Fresh_2 has two lifts and two reductions. □

Remark 7.7. Proposition 7.6 is why Fresh_2 is displayed in §1 rather than derived at leisure. It is not a gadget chosen for convenience: admissibility plus Theorem 3.3 leave nothing else of that size, and the only freedom remaining is the choice of the two templates, which Lemma 8.2 constrains to be injective with disjoint images.

8 The encoding

Definition 8.1 (Addresses)**.** An *address* is a name $\theta(n_0)$ with $\theta \in \{\mathsf{L}, \mathsf{R}\}^*$ and n_0 the static seed $@\prod_{x \in \mathrm{fn}(P)} x!(\mathbf{0})$.

We never write L or R in the image of the translation; they are metalanguage, used to *name* addresses the encoding produces by communication.

Lemma 8.2 (Injectivity and disjointness)**.** L *and* R *are injective up to* $\equiv_{\mathcal{N}}$*, their images are disjoint, and* $\mathrm{qd}(\mathsf{L}(\sigma)) = \mathrm{qd}(\mathsf{R}(\sigma)) = 1 + \mathrm{qd}(\sigma)$*. Hence* $\theta \mapsto \theta(n_0)$ *is injective on* $\{\mathsf{L}, \mathsf{R}\}^*$*.*

Proof. $@(\sigma!(\mathbf{0})) \equiv_{\mathcal{N}} @(\tau!(\mathbf{0}))$ requires $\sigma!(\mathbf{0}) \equiv \tau!(\mathbf{0})$, hence $\sigma \equiv_{\mathcal{N}} \tau$; similarly for R. A lift and an input are distinct constructors and $\equiv$ does not relate them, giving disjointness. Neither template yields a name of the form $@(*x)$, so quote depth strictly increases, which prevents an address from coinciding with a proper prefix of itself; injectivity of $\theta \mapsto \theta(n_0)$ then follows by induction on $|\theta|$. □

Lemma 8.2 realises Abramsky's bijection inside the rho calculus with n_0 as the unique root, and resolves prefix-incompatibility into two ingredients already present in the literature: stratification by quote depth separates addresses of different length, and template disjointness separates siblings.

Construction 8.3 (The allocator)**.**

$$\mathsf{Fresh}_2(\sigma;\, u,\, v).\, P \;\triangleq\; \mathsf{for}(\, u \leftarrow \sigma\,)\mathsf{for}(\, v \leftarrow \sigma\,)P \;\mid\; \sigma!(\sigma!(\mathbf{0})) \;\mid\; \sigma!(\mathsf{for}(\, y \leftarrow \sigma\,)\mathbf{0})$$

binding u, v in P. It is admissible; after two reductions $\{u, v\} = \{\mathsf{L}(\sigma), \mathsf{R}(\sigma)\}$.

That the assignment of $\mathsf{L}(\sigma)$ and $\mathsf{R}(\sigma)$ to u and v is nondeterministic is deliberate. Only distinctness is ever required, never identity; this is Lybech's criterion of parameter independence read as a design principle instead of a proof obligation, and Lemma 11.9 discharges it once for the whole development.

Definition 8.4 (The translation)**.** $[\![P]\!]$ is an *address abstraction*, a function from a name to a process; we write $[\![P]\!]_\sigma$ for its value.

$$\begin{aligned}
[\![\mathbf{0}]\!]_\sigma &= \mathbf{0} \\
[\![\bar{x}\langle z\rangle]\!]_\sigma &= x!(*z) \\
[\![x(y).P]\!]_\sigma &= \mathsf{for}(\, y \leftarrow x\,)[\![P]\!]_\sigma \\
[\![P_1 \mid P_2]\!]_\sigma &= \mathsf{for}(\, m \leftarrow \sigma\,)[\![P_1]\!]_m \;\mid\; \mathsf{for}(\, m \leftarrow \sigma\,)[\![P_2]\!]_m \;\mid\; \sigma!(\sigma!(\mathbf{0})) \;\mid\; \sigma!(\mathsf{for}(\, y \leftarrow \sigma\,)\mathbf{0}) \\
[\![(\nu z)P]\!]_\sigma &= \mathsf{Fresh}_2(\sigma;\, z,\, m).\, [\![P]\!]_m \\
[\![!\, x(y).P]\!]_\sigma &= \mathsf{Fresh}_2(\sigma;\, a,\, b).\, \mathsf{Fresh}_2(b;\, s,\, c).\, \big(\; \mathsf{D}(a) \;\mid\; a!(B) \;\mid\; s!(*c) \;\big) \\
\text{where}\quad B &= \mathsf{for}(\, y \leftarrow x\,)\Big(\mathsf{D}(a) \;\mid\; \mathsf{for}(\, c' \leftarrow s\,)\; \mathsf{Fresh}_2(c';\, d,\, e).\, \big(s!(*e) \mid [\![P]\!]_d\big)\Big)
\end{aligned}$$

Top level: $[\![P]\!] \triangleq [\![P]\!]_{n_0}$.

The replication clause. The node splits its address twice, obtaining a box channel a, a local server port s and a first seed c. The box B is posted on a and duplicated in the self-reproducing manner, so $\mathsf{D}(a) \mid a!(B) \to B \mid a!(B)$, whereupon B blocks on x: unfolding is input-guarded and there is no divergence. Each unfolding reinstalls a duplicator, takes the current seed c' off the local port, splits it into a body address d and a successor seed e, returns e to the port, and runs the body at d.

What is inside the quote is the point. By the time B is posted, a, s and x have been substituted, since they are bound by inputs whose continuations include the lift, and by Lemma 3.7 substitution reaches a lift's object. Thereafter B is copied verbatim and needs no renaming, because it contains no addresses — only the port on which it asks for one. This is the shape of Copy's first clause: the copies share the parent's port, and the routing does the distinguishing.

Remark 8.5 (Locality). The port s is fixed and shared — among the copies of *one* replication node, and derived from *that node's* address. A term with k replication nodes has k mutually unaware local servers. §9 makes this precise.

Remark 8.6 (Whole-term compilation survives). Every process above is written by the translation. What became dynamic is the *value* of an address, not the structure computing it: the tree of Fresh_2 nodes is static and isomorphic to the parse tree.

Remark 8.7 (Joins). In a calculus with joins the two receipts of Fresh_2 may be written $\mathsf{for}(\, u \leftarrow \sigma \,\&\, v \leftarrow \sigma\,)P$, which consumes both messages in one rendezvous and removes the nondeterminism entirely. We do not assume joins in the theory — the pure calculus does not have them — but every implementation does, and Appendix C uses the join form.

9 Locality: why this is not a distributed name server

The obvious objection to §8 is that a name server per replication node is still a name server. The objection is answered by observing what the images actually communicate on.

Definition 9.1 (Allocation site, name-traffic graph)**.** An *allocation site* of an encoding $[\![-]\!]$ is an occurrence in $[\![P]\!]$ of a subterm whose reduction produces a name not in $\mathcal{N}ms([\![P]\!])$. The *name-traffic graph* $G([\![P]\!])$ has the allocation sites as vertices, with an edge between two sites when a name that is the subject of a communication at one is also the subject of a communication at the other.

Theorem 9.2 (Locality)**.** *For every π term P:*

(i) every subject occurring in $[\![P]\!]_{n_0}$ is either in $\varphi(\mathrm{fn}(P))$, or is generated by a ν site, or is an address;

(ii) each address is the subject of communications belonging to exactly one occurrence of a clause of Definition 8.4;

(iii) the address that is the subject of an allocation is the one allocated by the parent of the allocating node.

Proof. (i) By induction on P: the clauses introduce no subjects beyond x, σ, and names bound by their own inputs, and the latter are either π names received on x, or addresses received on σ.

(ii) Assign to each node of P the address at which its clause is invoked. The clauses for $\mathbf{0}$, output and input allocate nothing and pass their address to the continuation, so an address may be shared along a prefix chain but is used as a subject only by the unique $|$, ν or ! node terminating that chain. Each of $|$, ν and Fresh_2 uses its address σ as a subject and allocates exactly $\mathsf{L}(\sigma)$ and $\mathsf{R}(\sigma)$, which by Lemma 8.2 lie strictly below σ in the address tree, so the subtrees allocated at $\mathsf{L}(\sigma)$ and $\mathsf{R}(\sigma)$ are disjoint. For !, the successive seeds are $c_0 = c$ and $c_{k+1} = e_k$ where d_k, e_k are the two children of c_k; the body of the k-th unfolding therefore occupies the subtree below d_k, and these are pairwise disjoint and disjoint from every c_j. (Note that the successor seed must be a *sibling* of the body address: advancing by $\mathsf{L}(c_k)$ while the body ran at c_k would make the next seed collide with the body's own first child.)

(iii) Immediate from the clauses: an allocating node's Fresh_2 or split has as subject the parameter it was passed. □

Corollary 9.3 (No shared allocation channel)**.** *$G([\![P]\!])$ is isomorphic to the tree of allocating nodes of P. In particular no name is the subject of allocations at two distinct nodes; there is no*

name v such that every allocation in $[\![P]\!]$ is a communication on v; and the diameter of $G([\![P]\!])$ grows with the depth of P.

Proof. By Theorem 9.2(ii) each address belongs to one node, and by (iii) the only sharing is between a node and its parent, which is the tree structure. A name v as in the statement would be the subject of allocations at every ν node, contradicting (ii) as soon as P has two. □

Remark 9.4. The same measurement applied to a server-based encoding gives a star: the server channel v is free in the image of every ν, so G has an edge from each ν site to the server and its diameter is 2 regardless of P. This is the difference between the two designs stated as a property of the images rather than as a matter of emphasis, and it is decidable by inspection. It is also, we think, the correct formal content of Lybech's own observation that a star does not represent the structure of a distributed source.

Corollary 9.5 (Freshness is confined to translation time)**.** *No reduction of $[\![P]\!]$ consults a process that is not emitted by the clause performing the reduction or by its parent; and the only name in $[\![P]\!]$ not derived by reduction from another is n_0, which is computed by the translation from* $\mathrm{fn}(P)$.

Remark 9.6. Abramsky's construction is not free of a freshness assumption either: his name convention stipulates a distinct root at each instance of Axiom, With and Of Course. What it buys, and what the above inherits, is that freshness is required only statically and finitely often. At runtime there is no allocation from a supply — only address extension, performed locally, paid for at the rate fixed by Theorem 3.3.

10 The counterexamples, worked

We guard each replication with an input on a trigger channel w and supply triggers from the context.

Example 10.1 (Example 5.1, guarded). $P_1' = !\, w(t).\big((\nu z)\bar{u}\langle z\rangle \mid Q\big)$ and $P_2' = (\nu z)\, !\, w(t).\big(\bar{u}\langle z\rangle \mid Q\big)$, separated in π by $[\,\cdot\,] \mid \bar{w} \mid \bar{w} \mid u(n_1).u(n_2).(\bar{n}_1 \mid n_2.\bar{x})$.

In $[\![P_1']\!]$ the k-th unfolding runs its body at d_k, and the body's parallel composition allocates $\mathsf{L}(d_k), \mathsf{R}(d_k)$ by communication on d_k, whose value differs with k by Theorem 9.2(ii); the enclosed ν therefore emits a different name at each unfolding. In $[\![P_2']\!]$ the ν is outside the box, its Fresh_2 fires once, and every unfolding emits the same z. The offending static increment has become a communication, and the value it computes tracks the unfolding.

Example 10.2 (Example 5.2, guarded). $P_1 = !\, w(t).(\nu z)\bar{u}\langle z\rangle$ and $P_2 = !\, w(t).(\nu q)(\nu z)\bar{u}\langle z\rangle$, with $P_1 \equiv P_2$. Unfolding k of $[\![P_1]\!]$ runs $\mathsf{Fresh}_2(d_k; z, m).u!(*z)$ and emits a child of d_k; unfolding k of $[\![P_2]\!]$ runs $\mathsf{Fresh}_2(d_k; q, m).\mathsf{Fresh}_2(m; z, m').u!(*z)$ and emits a grandchild. In both the emitted names are pairwise distinct across k, since the d_k are, and in neither is any emitted name observable except as a π name; the two are identified, as they must be.

11 Full abstraction relative to encoded contexts

11.1 Context-labelled bisimilarity

Definition 11.1. For a calculus with reduction $\to$ and contexts $\mathscr{C}$, write $P \xrightarrow{C} P'$ when $C[P] \to P'$, and $P \stackrel{C}{\Longrightarrow} P'$ when $C[P] \to^* P'$. For $\mathcal{D} \subseteq \mathscr{C}$, a symmetric $\mathcal{R}$ is a $\mathcal{D}$-bisimulation if $P\ \mathcal{R}\ Q$

and $P \xrightarrow{C} P'$ with $C \in \mathcal{D}$ imply $Q \xRightarrow{C} Q'$ with $P' \mathcal{R} Q'$; $\approx_{\mathcal{D}}$ is the largest such.

On the source $\mathcal{D} = \mathscr{C}_\pi$, written $\approx_\pi$; on the target $\mathcal{D} = [\![\mathscr{C}_\pi]\!]$, the encodings of source contexts.

Remark 11.2 (On minimality). The labels are enabling contexts, not *minimal* enabling contexts. Minimality — relative pushouts, and their refinement to a subcategory of admissible observers — is the right long-run discipline and is developed elsewhere; nothing below needs it. Without minimality the label set is larger and $\approx_{\mathcal{D}}$ correspondingly finer, which makes completeness harder rather than easier.

Definition 11.3. $[\![-]\!]$ extends to contexts with $[\![[\,\cdot\,]]\!]_\sigma$ a hole expecting an address; plugging is instantiation of the address abstraction.

Lemma 11.4 (Compositionality)**.** $[\![C[P]]\!] = [\![C]\!][[\![P]\!]]$*, on the nose.*

Proof. Induction on C: each clause of Definition 8.4 mentions its sub-translations only under an address abstraction, and the hole occurs in exactly one. □

11.2 Name usage and relabelling

The next three results are the apparatus the bisimulation argument rests on. The first is the syntactic induction that makes the informal claim "encoded contexts do not inspect names" precise.

Lemma 11.5 (Name usage)**.** *Let y be a variable bound in $[\![P]\!]_\sigma$ by an input arising from a π input, or by a Fresh_2 in the position of a ν-bound name. Then every occurrence of y in $[\![P]\!]_\sigma$ is either the subject of an input or of a lift, or is the z in a subterm $x!(*z)$. In particular no occurrence of y lies inside a quote, and no subterm $*y$ occurs except under a lift as above.*

Proof. Induction on P. For $\bar{x}\langle z\rangle$ the image is $x!(*z)$, in which x is a subject and z is as described. For $x(y).P$ the image is $\mathsf{for}(\, y \leftarrow x\,)[\![P]\!]_\sigma$ and the claim for y follows from the hypothesis for P. For $(\nu z)P$ the image is $\mathsf{Fresh}_2(\sigma; z, m).[\![P]\!]_m$, and z occurs only in $[\![P]\!]_m$. For $|$ and for $!$ the bound variables introduced are addresses, not π names, and are used only as subjects or in lift objects by construction (admissibility, Definition 7.4). □

Definition 11.6 (Address relabelling)**.** For addresses σ, τ, let $\beta_{\sigma\to\tau}$ be the map on names sending $\theta(\sigma) \mapsto \theta(\tau)$ for $\theta \in \{\mathsf{L}, \mathsf{R}\}^*$ and fixing all other names. It is a bijection by Lemma 8.2, and extends to processes by metalevel replacement.

Remark 11.7. $\beta_{\sigma\to\tau}$ is *not* a rho substitution: it must descend into the interiors of quotes, and by Lemma 3.7 no substitution does. It is available to the metatheory and to no rho context. This is a compact statement of why the encoding is safe from encoded observers, and §11.5 is a statement of why it is not safe from arbitrary ones.

Lemma 11.8 (Equivariance)**.** $\beta_{\sigma\to\tau}([\![P]\!]_\sigma) = [\![P]\!]_\tau$ *whenever σ, τ satisfy the distinctness condition of Theorem 9.2.*

Proof. Induction on P, using that every occurrence of an address in $[\![P]\!]_\sigma$ is either σ itself or a bound variable later instantiated to $\theta(\sigma)$, and that β commutes with L and R by construction. □

Lemma 11.9 (Address independence)**.** $[\![P]\!]_\sigma \approx_{[\![\mathscr{C}_\pi]\!]} [\![P]\!]_\tau$ *for σ, τ as above.*

Proof. Take $\mathcal{R} = \{(T, \beta_{\sigma\to\tau}(T))\}$ for T reachable from $[\![P]\!]_\sigma$. Lemma 11.8 gives the base case. For the step, β is a bijection on names, so it preserves and reflects $\equiv_{\mathcal{N}}$ and hence the applicability of the reduction rule; and for the labels, Lemma 11.5 shows that an encoded context uses a name it has received only as a subject or as the object of a lift of the form $x!(*z)$, so the only discrimination it can make between two received names is an equality test by attempted synchronisation, which a bijection preserves and reflects. In particular no encoded context applies $*\cdot$ to a received name, which is the only operation that could observe a name's structure. □

Corollary 11.10 (The allocator race is harmless)**.** *The two reductions of a* Fresh_2 *commute up to* $\approx_{[\![\mathscr{C}_\pi]\!]}$*: the two outcomes differ by* β *exchanging* $\mathsf{L}(\sigma)$ *and* $\mathsf{R}(\sigma)$ *and their subtrees.*

11.3 Administrative reductions

Lemma 11.11 (Administrative reductions terminate)**.** *Call a reduction* administrative *if its subject is an address. Every maximal administrative sequence from* $[\![P]\!]_\sigma$ *is finite, and no administrative reduction is observable on* $\varphi(\mathrm{fn}(P))$*.*

Proof. Each | node contributes two, each ν two, each ! four plus, per unfolding, one on a and three on s and c_k; unfoldings are guarded by an input on the source channel, so the number of administrative steps enabled is bounded by a function of the number of source communications performed. Unobservability is Theorem 9.2(i) together with the choice of n_0. □

Corollary 11.12 (Divergence reflection)**.** $[\![P]\!] \to^\omega$ *implies* $P \to^\omega$*.*

11.4 Operational correspondence and full abstraction

Proposition 11.13 (Completeness)**.** *If* $P \to P'$ *then* $[\![P]\!]_\sigma \to^+ T$ *with* $T \approx_{[\![\mathscr{C}_\pi]\!]} [\![P']\!]_{\sigma'}$ *for some address* σ'*.*

Proposition 11.14 (Soundness)**.** *If* $[\![P]\!]_\sigma \to^* T$ *then* $P \to^* P'$ *for some* P' *with* $T \approx_{[\![\mathscr{C}_\pi]\!]} [\![P']\!]_{\sigma'}$*.*

Theorem 11.15 (Full abstraction)**.** $P \approx_\pi Q \iff [\![P]\!] \approx_{[\![\mathscr{C}_\pi]\!]} [\![Q]\!]$.

Proposition 11.16. *Theorem 11.15 implies Lybech's criteria 1–6, with* $\simeq$ *taken to be* $\approx_{[\![\mathscr{C}_\pi]\!]}$ *and the name derivation function* $\sigma \mapsto \{\mathsf{L}(\sigma), \mathsf{R}(\sigma)\}$*.*

The proofs of Propositions 11.13, 11.14, 11.16 and Theorem 11.15 are deferred to Appendix B, which also sets out the induction and the case analysis they require. The apparatus they use is Lemmas 11.4, 11.5, 11.8, 11.9 and 11.11 and no more; in particular the nondeterminism of Fresh_2 enters only through Corollary 11.10.

11.5 The restriction is forced

Proposition 11.17. *There are* π *processes* $P \equiv Q$ *and a rho context* $C \notin [\![\mathscr{C}_\pi]\!]$ *with* $C[[\![P]\!]] \not\approx C[[\![Q]\!]]$*.*

Proof. Take $P = \bar{u}\langle v\rangle$ and $Q = \bar{u}\langle v\rangle \mid \mathbf{0}$. Then $[\![P]\!]_{n_0} = u!(*v)$, with no barb on n_0, whereas

$$[\![Q]\!]_{n_0} = \mathsf{for}(\, m \leftarrow n_0\,)u!(*v) \;\mid\; \mathsf{for}(\, m \leftarrow n_0\,)\mathbf{0} \;\mid\; n_0!(n_0!(\mathbf{0})) \;\mid\; n_0!(\mathsf{for}(\, y \leftarrow n_0\,)\mathbf{0})$$

has one. Since n_0 is computed by the translation from $\mathrm{fn}(P)$, an arbitrary rho context can name it: take $C = [\,\cdot\,] \mid \mathsf{for}(\, k \leftarrow n_0\,)succ!(\mathbf{0})$. No encoded context can, since $n_0 \notin \varphi(\mathrm{fn}(P))$. □

Remark 11.18 (The sharper form). A deeper version exploits reflection rather than scaffolding: an arbitrary rho context may apply $*\cdot$ to a name received on u, and since a ν-generated name here is $\mathsf{L}(\sigma)$ or $\mathsf{R}(\sigma)$ for an internal σ, dropping it runs $\sigma!(\mathbf{0})$ or $\mathsf{for}(\, y \leftarrow \sigma\,)\mathbf{0}$ and exposes a barb on an internal address. By Lemma 11.5 no encoded context does this. This is not a weakness of the present encoding: by Lybech's separation theorem the rho calculus manufactures new free — hence observable and substitutable — names at runtime, and no encoding of π into rho can be fully abstract against unrestricted rho observers. The restriction to $[\![\mathscr{C}_\pi]\!]$ is the shape of the theorem rather than a weakening of it.

Remark 11.19. Lybech's $\approx^{\mathcal{N}}$ restricts observation by stipulating a set of names; the restriction here is derived from the source. The two agree for π and rho, both being name-passing calculi for which barbs are an adequate observation. They do not agree in general: a name-set presentation assumes that what one can observe of a term is the set of names it can communicate on, which is false for, e.g., the ambient calculus.

12 Discussion

	name server	this note
implementation assumption	a global freshness oracle on a fixed channel	no runtime name source; a static seed
name-traffic graph	star, diameter 2	tree $\cong$ parse tree
translation	clause-by-clause	whole term
where freshness is required	at every ν, at runtime	at translation time only

The table compares design objectives and the implementation assumptions that follow from them, not degrees of correctness: both encodings are correct under their own assumptions, for the same source language, and Lybech's is proved in more detail than this one currently is.

Demand-driven copying is a dividend. Abramsky's Copy fires only when a contraction asks for a second copy. Restricting to input-guarded replication, which both encodings do to avoid divergence, is the process-calculus shadow of that discipline; the fix for divergence and the fix for freshness come from the same place.

Linearity was doing work we must now do by hand. In PE_2 the address invariant is protected by linearity — each name occurs exactly twice — and by acyclicity, which gives deadlock freedom. The rho calculus has neither. What was structural there is a theorem here, and its proof, Lemmas 8.2 and Theorem 9.2, decomposes along exactly Abramsky's two axes: depth and siblings.

13 Related work

Abramsky's programme was carried forward by Bellin and Scott [2] and tightened into a session-typed discipline by Caires and Pfenning [3] and Wadler [15]. Kokke, Montesi and Peressotti [8] observe that the separation of environments internalised by $\otimes$ corresponds to parallel process behaviour and extend linear logic to hyperenvironments accordingly: the same move as the present one, made from the side of the logic.

On encodability, Gorla [6] supplies the criteria Lybech adapts; Gorla and Nestmann [7] argue against full abstraction as a criterion; van Glabbeek [5] derives validity from a semantic equivalence rather than a list, which is closer in spirit to §11, though he does not treat parametrised translations. Sangiorgi's encoding of HOπ into π [13] is the result Lybech's separation theorem shows does not extend to reflection; the relabelling apparatus of §11 is the standard treatment of distinctions and indexed bisimulation [14] and is cited rather than reconstructed. Bendixen, Bojesen, Hüttel and Lybech [12] instantiate the rho calculus as a higher-order Ψ-calculus. Leifer and Milner [9] supply the derived-transition machinery a minimal-label refinement would use.

14 Open problems

1. **Minimal labels.** Replace enabling contexts by minimal ones, taken relative to $[\![\mathscr{C}_\pi]\!]$ rather than to all of rho, and show Theorem 11.15 survives with a coarser relation.

2. **A converse to Theorem 3.3.** The theorem bounds name creation from above. A matching lower bound for a class of allocation protocols would turn Proposition 7.6 into a genuine optimality result rather than a bound attained.

3. **Polyadic π and joins.** Encode polyadic synchronisation using the join form directly, connecting with Carbone and Maffeis [4].

4. **Choice.** The source fragment is choice-free.

5. **Cost.** The Fresh_2 tree is a metering surface: each source ν costs two communications, each unfolding four. Theorem 3.3 says the rate is optimal; what a whole term costs is not treated here.

6. **Mechanisation.** Definition 7.4 is a syntactic side condition and should be machine checked; a translation satisfying it cannot have a bug of the kind found in 2022.

Acknowledgements

The author thanks Stian Lybech, whose counterexamples are correct and whose diagnosis of their common cause is the reason the present repair can be stated briefly. Claude (Anthropic) assisted with source review, drafting, and proof-checking.

References

[1] S. Abramsky. *Computational interpretations of linear logic.* Theoretical Computer Science 111(1–2):3–57, 1993.

[2] G. Bellin and P. J. Scott. *On the π-calculus and linear logic.* Theoretical Computer Science 135(1):11–65, 1994.

[3] L. Caires and F. Pfenning. *Session types as intuitionistic linear propositions.* CONCUR 2010, LNCS 6269, 222–236.

[4] M. Carbone and S. Maffeis. *On the expressive power of polyadic synchronisation in π-calculus.* Nordic Journal of Computing 10(2):70–98, 2003.

[5] R. van Glabbeek. *A theory of encodings and expressiveness.* FoSSaCS 2018, 183–202.

[6] D. Gorla. *Towards a unified approach to encodability and separation results for process calculi.* Information and Computation 208(9):1031–1053, 2010.

[7] D. Gorla and U. Nestmann. *Full abstraction for expressiveness: history, myths and facts.* Mathematical Structures in Computer Science 26:639–654, 2014.

[8] W. Kokke, F. Montesi and M. Peressotti. *Better late than never: a fully abstract semantics for classical processes.* Proc. ACM Program. Lang. 3(POPL), 2019.

[9] J. J. Leifer and R. Milner. *Deriving bisimulation congruences for reactive systems.* CONCUR 2000, LNCS 1877, 243–258.

[10] S. Lybech. *Encodability and separation for a reflective higher-order calculus.* EXPRESS/SOS 2022, EPTCS 368, 95–112.

[11] L. G. Meredith and M. Radestock. *A reflective higher-order calculus.* Electronic Notes in Theoretical Computer Science 141(5):49–67, 2005.

[12] A. R. Bendixen, B. B. Bojesen, H. Hüttel and S. Lybech. *A generic type system for higher-order Ψ-calculi.* EXPRESS/SOS 2022, EPTCS 368, 43–59.

[13] D. Sangiorgi. *From π-calculus to higher-order π-calculus — and back.* TAPSOFT 1993, LNCS 668, 151–166.

[14] D. Sangiorgi and D. Walker. *The π-calculus: a theory of mobile processes.* Cambridge University Press, 2001.

[15] P. Wadler. *Propositions as sessions.* Journal of Functional Programming 24(2–3):384–418, 2014.

A Status of results

Result	Status	Note
Thm. 3.3, Cor. 3.4	proved	
Thm. 3.5, Cor. 3.6	proved	
Lem. 3.7	proved	
Prop. 7.1, 7.2	proved	elementary
Cor. 7.3	proved	from §3
Prop. 7.6	proved	bound attained, not optimality; see open problem 2
Lem. 8.2	proved	uses Lybech's stratification lemma
Thm. 9.2, Cor. 9.3	proved	
Lem. 11.4	proved	
Lem. 11.5	proved	syntactic induction
Lem. 11.8, 11.9, Cor. 11.10	proved	
Lem. 11.11	proved	
Prop. 11.17	proved	
Prop. 11.13, 11.14	**deferred**	Appendix B
Thm. 11.15, Prop. 11.16	**deferred**	Appendix B

B Operational correspondence and full abstraction

This appendix is to be supplied. Its content is fixed by the apparatus of §11 and is set out here so that the shape of the remaining obligation is visible.

B.1 Substitution. The proposition that $[\![P\{z/y\}]\!]_\sigma = [\![P]\!]_\sigma\{z/y\}$ for y a π variable, by induction on P, using Lemma 11.5 to see that y occurs only in positions substitution reaches. This is where admissibility is used a second time.

B.2 Completeness. Induction on the derivation of $P \to P'$. Base case [π-COM]: the images of the redex are $\mathsf{for}(\, y \leftarrow x\,)[\![P_1]\!]_{\sigma_1}$ and $x!(*z)$, and one reduction gives $[\![P_1]\!]_{\sigma_1}\{z/y\}$, which is $[\![P_1\{z/y\}]\!]_{\sigma_1}$ by B.1. Congruence cases: for $|$, two administrative steps precede; for ν, a Fresh_2; for structural congruence, the case $P \mid \mathbf{0} \equiv P$ requires Lemmas 11.11 and 11.9. Replication: one box copy, one seed exchange, one Fresh_2, then the source communication. Address bookkeeping throughout by Lemma 11.9, which is what permits σ' to be any address satisfying Theorem 9.2.

B.3 Soundness. Induction on the length of the target reduction, with each step classified as administrative or as the image of a source communication; the classification is exhaustive because by Theorem 9.2(i) every subject is an address, a source name, or a ν-generated name. Administrative steps are absorbed by Lemma 11.11; the only non-confluence among them is the Fresh_2 race, handled by Corollary 11.10. The case needing care is a partially completed unfolding, in which a box has been copied but the seed exchange has not occurred.

B.4 Full abstraction. From B.2, B.3 and Lemma 11.4. Forward: exhibit $\{([\![P]\!], [\![Q]\!]) : P \approx_\pi Q\}$ closed under $\approx_{[\![\mathscr{C}_\pi]\!]}$ as a $[\![\mathscr{C}_\pi]\!]$-bisimulation. Backward: contrapositively, a separating source

context encodes to a separating label, and the distinguishing barb is on a name of $\varphi(\mathrm{fn}(P) \cup \mathrm{fn}(Q))$, preserved and reflected by Lemma 11.11 and Theorem 9.2.

B.5 Lybech's criteria. Compositionality is Lemma 11.4; substitution invariance is B.1; operational correspondence is B.2 and B.3; observational correspondence follows from Theorem 9.2(i); divergence reflection is Lemma 11.11; parameter independence is Lemma 11.9.

C The encoding in rholang

```
// Addresses:  L(s) = @{ s!(Nil) }        R(s) = @{ for (y <- s) { Nil } }

// Fresh2(s; u, v).P   --- monadic form, as in the calculus:
//   for (u <- s) { for (v <- s) { P } }
//   | s!( s!(Nil) ) | s!( for (y <- s) { Nil } )
//
// Fresh2(s; u, v).P   --- join form, as in rholang (deterministic):
//   for (u <- s & v <- s) { P }
//   | s!( s!(Nil) ) | s!( for (y <- s) { Nil } )

// [[ P1 | P2 ]]_s
//   for (m <- s) { [[P1]]_m } | for (m <- s) { [[P2]]_m }
//   | s!( s!(Nil) ) | s!( for (y <- s) { Nil } )

// [[ (nu z) P ]]_s
//   Fresh2(s; z, m). [[P]]_m

// [[ ! x(y).P ]]_s
//   Fresh2(s; a, b). Fresh2(b; srv, c).
//     ( D(a) | a!( B ) | srv!( *c ) )
//   B = for (y <- x) {
//         D(a) |
//         for (cp <- srv) {
//           Fresh2(cp; d, e).( srv!(*e) | [[P]]_d )
//         }
//       }
//   D(a) = for (q <- a) { *q | a!(*q) }
```

The join form of Fresh_2 consumes both messages in one rendezvous and so has no race; it is what an implementation should use, and it is the concrete payoff of §2.1(2). The theory above assumes only the monadic form, and pays for that assumption in Corollary 11.10.